When the Monitor Blinds Itself: Observation-Bounded Failure Detection in Multi-Agent LLM Systems

ANONYMOUS AUTHOR(S)

In budget-bounded multi-agent LLM execution, detecting that a plan has gone stale mid-run is bounded by observation rather than by monitor intelligence, and an uncalibrated "smart" monitor manufactures the very blindness it cannot survive: its own false-interrupt economy consumes the run lifetime in which re-observation would have occurred, because precision and recall draw on one finite budget. We establish this finding through three acts on our own system. We proposed Sentinel Protocol, which compiles an orchestrator's plan assumptions into typed, observable tripwire conditions at plan time so workers monitor by cheap pattern matching and interrupt for replanning, and we pre-registered frozen kill gates and ran a pilot to a verdict we were not free to renegotiate. The verdict killed most of the architecture: strict detection recall 35%, the judge tier falsified as an interrupt filter (median false-interrupt rate 1.0), and the efficiency claim killed on its own pre-committed break-even fit, the architecture wasting more than the batch baseline it was built to save. One claim survived: where invalidation signals recur on observed surfaces, compiled tripwires detect 3× faster than a cost-matched periodic-revalidation baseline. Trace archaeology, adversarially adjudicated against two external red-team reviews by a deterministic zero-LLM replay battery, located the mechanism as self-inflicted observation starvation, confirmed on identical seeds where baseline systems re-observed every starved surface (12/12). The redesign restores observation as the detector: side-channel deterministic probes, compiled category-blind from general change-shapes so the held-out test is genuine, validated by zero-LLM shadow replay before being built, and re-measured under fresh pre-registered gates with escrowed held-out injection categories. We additionally report that a naive anomaly-gated baseline outperformed the full architecture at zero false interrupts, contribute TripwireBench and its manipulation-qualification protocol, and report the pre-registration apparatus itself, exercised through a kill verdict on our own system, as an artifact.

Additional Key Words and Phrases: multi-agent LLM systems, runtime verification, self-adaptive systems, failure detection, interrupt policy, pre-registration, agent orchestration

> **[NEW · author review: proposed abstract close]** Under fresh pre-registration with escrowed held-out categories, the redesign (v2) detected 24/31 injected faults at a clean-run false-interrupt rate of zero, partially generalized to the held-out categories, detecting RESOURCE_BUDGET (3/5) where every passive baseline scored 0/5, while DEPENDENCY_VERSION was missed by every arm including v2 (0/5, a harness-wide miss), and eliminated v1's false-interrupt self-harm, yet failed its pre-registered efficiency bar at 55.5% clean overhead against a 12% cap, an OVERALL FAIL on economics. The result is therefore observation-bounded compiled monitoring that detects, generalizes, and stops injuring clean runs but does not yet pay for itself: a validated detection-and-generalization result together with a characterized economic failure that defines the v3 problem.

1 Introduction

Twelve seconds before the injected fault it existed to catch, the monitoring system destroyed its own sensor coverage. In one pilot cell, a field-shape tripwire fired noise on a healthy file-listing response; a judge model approved the escalation as genuine at 0.98 confidence, hallucinating a misreading of the listing; the orchestrator paused its workers and replanned; and the recompiled tripwire set for the revised plan covered none of the surfaces the original set had armed. The injection fired twelve seconds later into an unmonitored world. The run proceeded confidently and failed. Every stage of this sequence worked as implemented; the architecture as a whole manufactured its own blindness. This paper is about that mechanism and what it implies: in budget-bounded multi-agent execution,

2026. ACM 2994-970X/2026/7-ART1
https://doi.org/10.1145/nnnnnnn.nnnnnnn

detecting a stale plan is bounded by observation rather than by intelligence, and an uncalibrated monitor starves its own observation. We establish the finding through a pre-registered experiment that killed most of our architecture, an adjudicated diagnosis of why, and a redesign that restores observation as the detector.

The economic stakes are a multiplier, not a margin. Production multi-agent LLM systems consume roughly 15× the tokens of single-agent chat [27], and when a plan-invalidating event occurs mid-execution (a dependency renamed, an endpoint removed, a schema drifted, a retrieved fact contradicted) batch orchestration discovers it only at aggregation, after N parallel workers have spent the multiplier on a dead plan. Failures of this kind are systemic and taxonomizable [14, 16], and interrupt *mechanisms* exist in production frameworks [22]. What does not exist is the policy layer: something that decides what to watch for, whether an anomaly is plan-breaking, and when interruption pays.

We proposed such a layer. **Sentinel Protocol** compiles the orchestrator's plan assumptions into typed, observable tripwire conditions at plan time: expensive reasoning once, performed by an LLM constrained by a fixed failure ontology and a typed DSL, so that workers monitor by cheap pattern matching and validated invalidations interrupt the orchestrator for replanning. Viewed through the software-engineering lens, it is a MAPE-K loop [23] whose Monitor and Analyze components are *generated per-plan* rather than engineered by hand (§3).

We then did to this architecture what the field rarely does to its own proposals: we pre-registered kill gates, frozen thresholds with pre-committed failure branches, custody-pinned artifacts, manipulation-validated fault injections, and a numbered deviation log, and ran the pilot to a verdict we were not free to renegotiate. **The verdict killed most of the architecture.** Strict detection recall was 35% against a 60% bar and a 40% kill floor. The judge tier approved every one of the 34 escalations that reached it, including all 18 false ones: a rubber stamp, not a filter. Clean-run overhead breached its cap, and the pre-committed break-even fit later found that the v1 architecture wasted *more* than the batch baseline at any fan-out. One pre-registered claim survived: a 3× detection-latency advantage over a cost-matched heartbeat where signals recur (median 3 vs. 9 tool calls; bar 2×).

Negative verdicts are only useful if their mechanism is found, so we performed trace archaeology over the fully-instrumented 195-run corpus and then: rather than grading our own homework, sent the diagnosis and redesign to two frontier-model red teams for blind adversarial review, and adjudicated every trace-testable attack empirically with a deterministic replay battery at zero LLM cost. The adjudicated diagnosis: detection in this regime is *observation-bounded*; a correct, armed monitor detects nothing on a surface nobody observes after the violation, and in our pilot that bound was **self-inflicted**. On identical seeds, baseline systems re-observed every starved surface (12/12); what starved the sentinel was its own noise economy, false interrupts consuming the run lifetime in which re-observation would have occurred. Monitoring precision and recall, in budget-bounded agent execution, are not independent dials; they share the run's finite lifetime.

The redesign follows mechanically from the trace evidence, and its central mechanism was validated *before being built*: a zero-LLM shadow replay compiled deterministic probes for all nine pilot injections and executed them against byte-identical replayed worlds, blocking 18/18 false interrupts under a probe-primary corroboration policy with projected strict recall of 21/27; **a projected ceiling under stated assumptions, not a result**. The redesigned architecture (v2) is being measured now under fresh pre-registered gates, frozen before any v2 data existed, with two held-out injection categories whose parameters are escrowed with a non-implementer: structurally unseen by the design. *[PENDING: Phase 1b confirmatory results, governed by schedule gate 2 (verdict due 2026-07-18); §7 carries the frozen protocol now and receives the results table verbatim when the gates compute.]*

> **[NEW · author review]** The confirmatory verdict is in hand: the redesign cleared detection and noise and failed economics. v2 detected 24/31 injected faults (recall 10/15 = 66.7% on the recoverable class) at a clean-run false-interrupt rate of zero, partially generalizing to the two escrowed held-out categories: it detected the held-out RESOURCE_BUDGET (3/5) where every passive baseline (batch S1 and heartbeat S3) scored 0/5, while the second held-out category, DEPENDENCY_VERSION, was missed by every arm including v2 (0/5); because that miss was harness-wide (no monitor detected it), it reads as a detection/benchmark limit rather than a v2-specific generalization failure (§7), and breached its 12% clean-overhead cap at 55.5%, an OVERALL FAIL driven entirely by economics (§7). The thesis survives in a sharper form: restoring observation fixes detection and the noise economy, but in budget-bounded execution the restored observation must also be *cheap*, and v2's is not.

Contributions. This paper makes one finding and contributes the apparatus that establishes and supports it.

(1) **The finding: detection in this regime is observation-bounded, and the bound is self-inflicted.** A correct, armed monitor detects nothing on a surface nobody re-observes after the violation, and in budget-bounded execution an uncalibrated monitor spends on false interrupts the lifetime re-observation needs, so precision and recall couple through one finite budget. Two companion regularities frame it: compilation is information-bounded, and the precision/recall coupling is the mechanism that turns a credulous filter into observation starvation (§6).

The finding is established and made useful by:

(2) **A pre-registered negative result that proves the smart design fails** (§5): frozen kill gates with pre-committed failure branches killed our own flagship architecture, with recall 35% against a 60% bar, a judge tier that approved all 18 false escalations, and economics inverted against the batch baseline.

(3) **A redesign that follows mechanically from the diagnosis** (§3, §6): side-channel deterministic probes, compiled category-blind from general change-shapes, restore re-observation as the detector and were validated by zero-LLM shadow replay before construction.

(4) **TripwireBench** (§4): an injection benchmark with manipulation-validated faults, oracle-recovery labeling, escrowed held-out categories, and waste/latency/noise metrics, built to make the finding falsifiable.

(5) **The surviving result and the baseline the field should know** (§5): compiled tripwires detect 3× faster than a cost-matched heartbeat where signals recur, and naive anomaly-gated escalation detected 56% at a false-interrupt rate of 0.0, beating the full architecture in every category.

(6) **A pre-registration and kill-gate methodology for agent-systems research** (§§5–7), exercised through a kill verdict on its authors' own architecture and extended with binding external red-team adjudication, data embargo, dev-run ledger, escrowed holdouts, and a category-blind compiler whose generalization test is structurally genuine.

> **[NEW · author review]** With Phase 1b resolved, the contribution is no longer "one finding plus a pending redesign" but **a validated detection-and-generalization result together with a characterized economic failure that defines the v3 problem**: (i) observation-bounded compiled monitoring, restored as active side-channel probes, **detects 24/31 injected faults at zero clean-run false interrupts** and **partially generalizes to the held-out categories**, detecting RESOURCE_BUDGET (3/5) where every passive baseline scored 0/5, while the second held-out, DEPENDENCY_VERSION, was missed by every arm including v2 (0/5), a harness-wide miss that reads as a detection/benchmark limit rather than a v2-specific generalization failure (§7); (ii) the same architecture's **runtime economics fail the pre-registered 12% efficiency bar at 55.5% clean overhead and miss heartbeat waste-parity at 1.09×**, OVERALL FAIL; (iii) the negative economic result, with its cost autopsy, is itself the contribution that frames v3, strong,

> generalizing, quiet detection that does not yet pay for itself. The pre-registration apparatus again bound the authors: the gates returned FAIL on the authors' own redesign and the branches executed as written.

2 Background and Related Work

Self-adaptive systems. SE readers will correctly recognize the loop: MAPE-K has organized self-adaptation since Kephart and Chess [23], with two decades of elaboration [24] including proposals to embed LLMs within the loop [25, 26]. We do not claim the loop. The claimed novelty is the provenance of its components: the Monitor and Analyze tiers are *compiled per-plan at runtime* by an ontology-constrained LLM, rather than engineered at design time, to our knowledge no prior MAPE-K instantiation generates its own Monitor specification from the plan it is monitoring. The pilot then demonstrated empirically that the Analyze tier is the hard part: an uncalibrated filter does not merely fail to add value, it destroys it (§5).

Plan-assumption runtime verification: the nearest ancestor, confronted. RVPLAN [6, 34] mechanically translates STRIPS/PDDL action preconditions into Past (FO-)LTL monitors compiled via DEJAVU, detecting violations of the planner's assumptions at execution time with replanning as the intended consumer; the paradigm has still-earlier roots in on-board-autonomy assumption monitoring [35] and the V&V-meets-planning frame [36], and remains active [37]. *Monitor the plan's assumptions, interrupt for replanning* is therefore established prior art. Sentinel Protocol ports the paradigm to a regime where each of its three enabling assumptions fails. **First, the specification does not exist.** RVPLAN's compilation is linear-time syntactic translation of preconditions already sitting machine-readable in a domain file; our compiler must extract and invent the load-bearing assumptions of a natural-language plan under an ontology prior: synthesis where no formal spec exists, with information-bounded coverage as the measured consequence (§6). **Second, the world does not narrate itself.** RVPLAN's monitors consume a benevolent perception stream of ground propositions in the monitor's own vocabulary; ours observe raw, unlabeled, single-visit tool telemetry, and the pilot's central result concerns precisely the observation gap that setting assumes away. **Third, the signal stream is noisy.** In a deterministic propositional stream a precondition either holds or it does not: RVPLAN's monitors cannot false-fire, so nothing needs filtering, judging, or pricing; in our regime the interrupt-economics layer is where the architecture lives, and where its first version measurably failed. RVPLAN's evaluation is monitor-synthesis timing on one rover example; TripwireBench contributes manipulation-validated injections and recall/noise/latency/waste measurement. The confrontation strengthens the contribution: an established paradigm breaks on contact with the LLM-agent regime in three measurable ways, and this work characterizes what surviving there takes.

Runtime verification and monitor synthesis [4, 5] supply the spec-to-monitor compilation tradition; we trade full temporal logic for a predicate DSL over discrete tool-call events. The pilot indicts the DSL's *passivity*, not the predicate trade-off; the v2 probe extension (§3) restores the RV tradition's insistence that monitors actually observe.

Guardrails and enforcement for LLM agents (AgentSpec [9], Agent-C [10], Pro2Guard [11], ProbGuard [12], LlamaFirewall [13]) enforce *externally authored* constraints, predominantly for safety. Our deltas: provenance (constraints compiled from the plan itself), objective (plan validity and efficiency, not action safety), and response (validated interrupts feeding replanning). Weak-to-strong monitoring [8] supports selective filtering over exhaustive review.

Failure taxonomies. MAST [14] taxonomizes agent-endogenous failure from 1,600+ traces; SHIELDA [20] catalogs agent-level exceptions. Our ontology covers the world-exogenous complement: the agents behave; the world the plan assumed has changed, and is used *generatively*, as the inductive bias for compiling monitors before execution rather than classifying failures after it (§3).

Observability and diagnosis. AgentOps [15], LumiMAS [17], wasted-computation diagnosis [16], and DiLLS [18] instrument and explain failures, largely post-hoc, for humans; Sentinel Protocol acts on the same signals in-flight, with the orchestrator as consumer. The wasted-computation line independently validates the premise that post-invalidation burn is a first-order cost.

Orchestration and interruptible execution. LangGraph's `interrupt()` [22] is mechanism without policy; escalation-ladder schedulers [19] consume validated interrupts; production economics are documented in [27]. We occupy the layer between: deciding *what to watch* and *whether to pull the trigger*.

Positioning in one sentence. Sentinel Protocol is an interrupt-policy compiler and filter for multi-agent LLM systems, occupying the layer between interrupt mechanisms (which exist) and orchestration recovery protocols (which exist), neither of which decides what to watch for. The pilot adds an empirical boundary to the positioning: the layer's value concentrates in *detection latency* (validated at 3× against a cost-matched heartbeat) and is destroyed by *uncorroborated interruption* (falsified: judge-approved noise starved recall and injured clean runs). Under the redesigned corroboration policy the interrupt stream becomes nearly vestigial: the shadow replay suppressed 15 of 16 true interrupts and recall recovered through the probe channel instead, so the architecture's honest identity is narrower and sharper than originally claimed: **scheduled deterministic verification compiled from the plan, with a status-coded interrupt fast lane**.

[AUTHOR: v6.1 §9.8 cites the authors' two prior ontology-constrained systems; both citations are unpublished and identity-revealing. Standard double-anonymous practice is to omit the paragraph or cite an anonymized supplement: decide at submission.]

3 Sentinel Protocol

This section presents the architecture in its current (v2) form, the form under confirmatory evaluation, and identifies, where the pilot requires it, how the pre-registered predecessor (v1) differed. v2 is not a fresh design: every difference from v1 cites the trace evidence that forced it (§6), and v1's verdict is reported in full regardless of v2's outcome (§5, §7).

3.1 Tiers

Tier 1: Orchestrator. Holds the global strategy: plans, dispatches, and is the sole entity authorized to replan. It receives only corroborated invalidations; on interrupt it pauses the affected scope, ingests the evidence, replans, and redispatches, with a fresh tripwire compilation for the revised plan. *(Pilot: functioned as designed; replan quality, not mechanics, was the weak link.)*

Tier 2: Sentinel (compile-time). Receives the plan and synthesizes typed, observable tripwire conditions: expensive reasoning once, amortized across execution. Compilation is constrained by a fixed failure ontology (§3.3) and a typed DSL (§3.2). *(Pilot: substantially validated, 89% paper coverage of planned injections; coverage misses across 27 injected cells were approximately zero once a coverage-heuristic artifact was corrected.)*

Tier 3: Workers. Execute tasks carrying a tripwire checklist; monitoring reduces to pattern matching against pre-compiled conditions, with no global reasoning. On a match they escalate with a structured evidence payload.

The information flow is a six-phase cycle: (1) plan, orchestrator → sentinel; (2) compile, sentinel → workers (task + typed checklist); (3) execute, with local matching; (4) escalate, worker → corroboration layer, carrying evidence; (5) interrupt, only corroborated invalidations reach the orchestrator, each with a recommended action; (6) replan, pause the affected scope, revise, redispatch with fresh tripwires. The design principle: pay expensive inference once at compile time, keep the hot path cheap and deterministic. The pilot added its corollary, which v2 implements: the hot

path must also *observe*, cheap deterministic probes belong to the compiled output, not only passive predicates.

What v2 removes: the runtime judge. v1 interposed an LLM judge between escalation and interrupt. Its corpus record across the pilot: 34/34 escalations approved (including all 18 false ones), the only two genuine signals it ever down-ruled were ruled noise, and one hallucinated misreading was approved at 0.98 confidence. Per the pre-committed gate branch and a pre-commitment made before the adjudication battery's results were readable, **the two-tier (no-judge) configuration is the designated primary architecture**; a rebuilt judge survives only as an exploratory arm. There is no post-selection: the designation is in ink.

3.2 Tripwires, probes, and the DSL

A tripwire is a pre-compiled observable condition with four required properties: *observable* (checkable from tool outputs, status codes, schemas, hashes, content), *parameterized* (concrete values, never "if the API changed"), *actionable* (a typed replan hint), and *calibrated* (severity and scope). Severity (CRITICAL/WARNING/INFO) and scope (global/local) determine routing; severity assignment is governed by a two-clause adjudication principle established during the build's compilation-quality gate and preserved through the redesign: WARNING is defensible when the monitored data plausibly survives in recoverable form; CRITICAL is required when nothing recoverable remains or when continuing invites the agent to fabricate evidence or corrupt a decision. The DSL is a predicate language over discrete tool-call events: `status_in`, field regexes and pointers, order violations, contradiction triggers, deliberately trading full temporal logic [4, 5] for deployability over raw tool telemetry. One transmission rule, learned five separate times during the build and enforced since: every constraint the compiling model must obey lives in schema-visible field descriptions or enums, constraints expressed in comments or prose do not transmit to the compiler.

The pilot vindicated the predicate language and indicted its passivity, so v2's load-bearing extension is the **compiled active probe**: each tripwire may carry a probe (method, target, cadence, cost class) (a deterministic HEAD request, schema fingerprint, gate-status re-read, or premise re-fetch) compiled from the same mechanical sources as the tripwire and executed by the matcher layer with **no LLM inference on the probe path**. Three constraints, each forced by measurement:

- **Side-channel isolation.** Replay measured three concrete vectors by which naive probes perturb the world they measure (the benchmark's injection clock, a seed-derived token stream, a validation-gate sentinel); probes therefore execute on a perturbation-isolated side channel, with non-perturbation a hard confirmatory-study gate.
- **Event-gated cadence.** The naively pre-committed cadence cost up to 2,322 probe calls against a 107-call run, 16–22× the run's own traffic, while simultaneously starving short runs of any sweep at all; v2 gates cadence on events and work-at-risk, with a guaranteed pre-completion sweep, and books probe cost as waste, not as a separate forgivable line.
- **Probe-primary corroboration.** v1's drafted "second independent signal" clause is broken as written: correlated noise self-corroborates, 6 of 18 false interrupts passed it, two on a clean run. In v2 a non-status-coded signal can interrupt only with a confirming probe; the status-coded fast path is retained, validated in-corpus (zero false interrupts carried status ≥ 400).

> **[NEW · author review]** A content-shaped or value probe is inert without a clean reference, so each such probe diffs the post-violation surface read against an **arm-time side-channel clean baseline**: at run start, before any worker call, the conductor sweeps every load-bearing surface on the perturbation-isolated probe channel and records its earliest clean state, and an interrupt fires only when the post-injection

> read differs from that baseline. The confirmatory probe-validity audit verifies the ordering *baseline (arm-time) < injection < detection* on every probe-driven interrupt (seed-1102 audit: 10/10 targeted/fresh/non-perturbing/independent). The baseline is generated by the system itself, not borrowed from a same-seed replay: the live counterpart of the shadow replay's clean-world affordance (projection-optimism caveat, §10).

Each tripwire additionally carries an `evidence_class` (status-coded, content-shaped, field-shape, counter) with trust priors (field-shape evidence, the pilot's false-interrupt engine (14 of 18 false interrupts), starts low-trust) and a `recovery_class` hint from a five-value taxonomy (REMAP, REINTERPRET, REDO, RECONCILE, RETREAT) distilled from the pilot's five injection semantics. A priced ABORT path exists: when nothing recoverable remains or projected recovery cost exceeds remaining task value under the break-even inequality evaluated at runtime, the correct output is salvage-and-stop, counted separately so aborts cannot launder detection parity.

3.3 Failure ontology and calibration

Compilation is constrained by an eight-category failure ontology (DEPENDENCY_VERSION, API_SURFACE, SCHEMA_DRIFT, TOOL_CONTRACT, RETRIEVAL_INTEGRITY, PERMISSION_AUTH, QUALITY_GATE, RESOURCE_BUDGET) that converts "what could go wrong" into a bounded checklist traversal.

> **[NEW · author review]** The sentence above ("Compilation is constrained by an eight-category failure ontology…") describes **v1** and reads in the present tense; it should be past-scoped ("v1's compilation was constrained by…") so it does not contradict the **"v2 compiles category-blind"** paragraph that immediately follows, whose compiler receives no category list at all.

v2 compiles category-blind. Where v1 constrained compilation by traversing the named eight-category ontology, v2's compiler receives no category list at all. It reasons in two general primitives: a dependency-noticing prompt (what does this plan step trust about the world) and the six general change-shapes the DSL already expresses (a field vanished, a status moved, a structure changed, a value moved, an order scrambled, a relation broke), grounded by worked examples drawn only from the five seen categories and selected under a rule frozen before any prompt tuning. Category labels are applied after detection, at analysis time, and for the held-out two that labeling happens escrow-side. The consequence is methodological: the compiler's generalization to the held-out categories tests whether reasoning in general change-shapes transfers, rather than rewarding a lookup against a category it was handed (§7).

The pilot's sharpest ontology lesson is that categories differ enormously in *observability under passive monitoring*, decomposing by signal shape rather than category quality: the one passing category (PERMISSION_AUTH, 5/6) is precisely the one whose signal is loud, sustained, and status-coded, every post-expiry call re-observes the violation, while content-shaped, single-visit categories (RETRIEVAL_INTEGRITY 0/3, TOOL_CONTRACT 1/6) were invisible to passive predicates. The probe field exists to convert every category's signal shape into PERMISSION_AUTH's.

Three template sources keep compilation grounded without model creativity: tool contracts and skill specifications compile directly; API documentation and schemas compile mechanically into surface tripwires for every endpoint the plan touches (empirically load-bearing, mechanically supplying the environment surface lifted paper coverage from 67% to 89%); and historical outcomes retrieved from a temporal knowledge graph of tripwires, outcomes, and fix patterns [30]. A fourth source is the pilot's accidental discovery: the benchmark's own manipulation-check apparatus included cheap deterministic premise probes, and one of them caught an injection the entire compiled passive set was blind to; the validation machinery invented the monitor the compiler lacked, and v2 promotes that mechanism into the DSL.

Calibration closes the loop: per-category and per-evidence-class Beta posteriors over "an escalation here is genuine," updated by conjugate outcome counting (no reinforcement learning), set escalation thresholds with severity floors. In v1 this loop ran cold: every prior uninformative, the mechanistic root of the judge's credulity; the pilot's 195 fully-traced runs are its first real posterior data, and the v1→v2 delta is itself a reported result.

4 TripwireBench

Measuring an interrupt policy requires a world that breaks on schedule. TripwireBench's pilot tier is a fully mocked, deterministic environment: a local multi-service API world (auth, inventory, pricing, shipping, document store, fixture repository), seed-derived fixtures, no wall-clock values in any payload, in which injections are exact state mutations triggered at exact global tool-call counts, so byte-identical replay of any run is possible from its configuration. Four task archetypes (API aggregation, repository migration, research synthesis, document pipeline) are exercised by an orchestrator-worker harness over a production agent CLI; all tool traffic passes one middleware that counts calls, fires injections, evaluates armed tripwires by pure predicate matching, and writes a complete JSONL trace.

Counter-triggered injection, the same mutation at the same global call count across systems and seeds, is what makes batch and treatment comparable: both worlds break at the same point in the *work*, not the same point in time. Two injection designs were redesigned during calibration because the batch baseline initially shrugged them off, and both lessons generalized: a schema drift whose new field name self-described the change was silently absorbed (the rename target must not narrate the mutation), and a token expiry with a live refresh path was silently re-authenticated around (recovery must require an orchestrator-level decision, not a worker retry).

Injection categories. Five categories were injected in the pilot: endpoint removal (API_SURFACE), silent schema drift (SCHEMA_DRIFT), hard token expiry (PERMISSION_AUTH), document contradiction (RETRIEVAL_INTEGRITY), and a validation gate that silently stops checking while still admitting outputs downstream (TOOL_CONTRACT). Two further categories: RESOURCE_BUDGET (a metered quota whose loud signal arrives only after the budget is dead) and DEPENDENCY_VERSION (a silent minor version bump that truncates default pagination), were authored *after* the pilot as **held-out categories**: their parameter draws (seeds, fire counters, quota values, version/page-size pairs) are escrowed with a non-implementer and structurally unseen by the v2 design, so that part of the confirmatory study's recall denominator cannot be overfit.

Manipulation qualification. No injection enters the matrix until it demonstrably wounds the *batch baseline*: for each (task, injection) pair, injected batch runs must fail task validation (or emit wrong output) while clean runs pass, under a frozen rule, before any treatment system is measured. The pilot qualified 9/9 pairs; the held-out categories passed the same bar (RESOURCE_BUDGET: clean 3/3 pass, injected 3/3 fail; DEPENDENCY_VERSION: clean 3/3, injected 3/3 failing by the intended wound, after two specification revisions). The full-study rule adds an oracle-recovery requirement: every injection must admit a recovery path, or be explicitly labeled a *retreat-condition cell* measuring abort quality rather than recovery.

The held-out qualification record doubles as benchmark texture worth reporting, because the batch baseline defeated two category designs before the third qualified. *Plan-time pre-armoring defeats default-drift:* in every seed, the orchestrator wrote explicit pagination parameters into worker instructions before seeing any response, so a mutation that only moves a default page size is unobservable, and an in-body `total_count` field, included as the detection fingerprint, instead tipped workers off to re-paginate (the self-describing-mutation lesson, learned twice). *Single-visit behavior accidentally immunizes:* all host plans enumerate collections in their first calls, so a mid-run contract change lands after the only enumeration touch, deploy-time skew must fire at execution

start to bite. *Orchestrator consolidation:* the planner collapsed a seeded three-worker plan into a single worker in 5/6 runs, with the sole multi-worker plan producing the sole clean pass at the original turn cap, plan-shape variance is part of the measured system, deliberately not prompted away.

Systems under test. S1 batch (dispatch, wait, aggregate, one redo); S2 naive interrupt (workers escalate any anomaly directly to a self-adjudicating orchestrator: no compiler, no judge); S3 cost-matched heartbeat (orchestrator re-validates assumptions every k worker calls, k calibrated so heartbeat overhead equals the sentinel's measured compile+judge overhead; the strongest fair version of the obvious alternative); S4 sentinel-no-judge; S5 the full v1 architecture.

Task archetypes. API aggregation (workers query three services and reconcile a structured report), repository migration (a coordinated rename across a fixture repo gated by a validation endpoint), research synthesis (retrieval and cited brief-writing through a document-validation gate), and document pipeline (generate, validate, package through a required gate). Each task specifies a goal, a seeded plan shape, 4–8 ground-truth assumptions (the compiler's coverage targets), its eligible injections, and a programmatic exact-match success checker derived blind from the task specification.

Metrics. Wasted work (post-injection tokens and tool calls not contributing to the final plan, plus discarded partials, in tokens and USD); time-to-detect (global tool calls from injection to first attributable orchestrator pause); detection recall; false-interrupt rate (with 0/0 defined as 0, ruled a priori); clean-run overhead; programmatic task success: no LLM grading anywhere. Pause attribution is by written trace rule, with a mandated manual audit of a 20% sample plus every disagreement case; in the pilot the audit returned 13/13 agreement. The confirmatory study adds salvage score, abort quality, probe validity, and probe overhead booked as waste.

Custody and embargo. The benchmark's integrity machinery is part of the artifact: frozen pre-registration with SHA-256 custody pins, a numbered deviation log, counter-triggered injections only (the admin path is test-only), a dev-run ledger recording every execution involving any treatment component, a data embargo before gate freeze, and escrowed holdout parameters verified by a public hash. The full study wraps established suites (GAIA [31], tau-bench [32], SWE-bench Verified subsets) with the same protocol (§8).

5 The Pre-Registered Pilot and Its Verdict

Design. 13 task variants (9 qualified injected pairs + 4 clean) × 5 systems × 3 seeds = a 195-cell matrix, run to completion (195/195 banked; 220 queue jobs with every failure replaced; zero malformed trace lines) under pinned tooling: pinned CLI version with a mid-matrix halt guard, fixed model assignments per tier (a mid-size model for orchestration and compilation, a small model for workers and judging), isolated session environments, and a frozen pre-registration whose artifact hashes are pinned in commit history. Injections fire at 50% of each task's clean-median call count (floored, by a rule committed before the medians existed), identical across systems and seeds. A compilation-quality gate preceded the matrix: compiled tripwires were scored against a four-property rubric by two independent frontier-model raters (95.1% agreement, disagreements adjudicated under a written principle), passing at 89% paper coverage of planned injections against an 80% bar. Kill gates KG1–KG4 were frozen before the first run, each with a pre-committed branch for failure; the gates module computes once, on the complete matrix, and refuses partial computation. The window for the post-verdict decision was itself pre-committed: 48 hours from the gate table, "do not negotiate with the data".

Verdict (computed once, verbatim):

```
KG1 recall: 35% (>=60%) | categories >=50%: 1/5 (>=4)            -> FAIL
```

```
KG2 FIR: S5=1.0 S2=0.0 S4=1.0 (S5<=0.5*S2 and S5<=0.7*S4)        -> FAIL
KG3 cost: S5 med $1.178952 vs S1 med $0.340831; success S5=4%
     S1=22%; clean overhead OVER (<=12%)                          -> FAIL
KG4 vs heartbeat: wasted med S5=15014.0 S3=7950 (>=20% better)
     | TTD med S5=3 S3=9 (>=2x)                                   -> PASS (TTD arm)
```

The pre-committed branches executed as frozen: KG1 at 35% falls in the kill band (<40%); the v1 detection claim is killed and the original flagship plan dropped; KG2's branch reframes to a compile-only architecture; KG3's branch demands the break-even fit (§9), which subsequently killed the efficiency claim outright; KG4 passes on its time-to-detect arm only, compiled tripwires detected the injected invalidation in a median of 3 tool calls versus the cost-matched heartbeat's 9, while the wasted-work arm failed (median 15,014 vs. 7,950 tokens): **detection speed did not convert to savings under v1's interrupt machinery**.

> **[NEW · author review]** This section reports the **v1 pilot (Phase 1)** and its kill verdict as the motivation for the redesign; it is kept distinct from the **v2 confirmatory (Phase 1b) verdict**, now in hand and reported at equal prominence in §7. In one line, the two-study arc: v1 killed on detection, noise, and economics; v2 (deviation-finalized) **clears detection (1bKG1's four pre-registered detection sub-terms; the composite gate prints FAIL on a separate folded precondition, §7) and noise/self-harm (1bKG2) and fails economics (1bKG3) and heartbeat waste-parity (1bKG4), OVERALL FAIL**: strong detection that fails the efficiency bar.

Interpretation discipline. Detection means a judge-confirmed, trace-attributable orchestrator-level pause (strict reading, logged a priori before gates computed); the mandated attribution audit manually verified a 20% sample with 13/13 agreement. A pre-committed generous reading (counting non-pausing escalations) recovers only two cells: 11/27 vs. 9/27, so the recall failure is real sensor silence, not attribution strictness. One cell's injection never fired and is excluded by the gate denominator (9/26 = 35% as printed; 9/27 = 33% per-cell; identical verdict either way).

Per-category strict recall: PERMISSION_AUTH 5/6; the passing category, exactly where the signal is loud, sustained, and status-coded; API_SURFACE 2/6; SCHEMA_DRIFT 1/6; TOOL_CONTRACT 1/6; RETRIEVAL_INTEGRITY 0/3. Content-shaped, single-visit signals were invisible to passive status/path predicates; the predicted near-zeros were confirmed.

The naive baseline result, stated with full prominence. S2: workers escalate any observed anomaly directly to the orchestrator, which adjudicates raw evidence itself; no compiler, no tripwires, no judge, achieved strict recall of **15/27 (56%) at a median false-interrupt rate of 0.0**, beating or tying the full v1 architecture in *every* failure category (S2 vs. S5 per category: API_SURFACE 5/6 vs. 2/6; SCHEMA_DRIFT 2/6 vs. 1/6; PERMISSION_AUTH 6/6 vs. 5/6; TOOL_CONTRACT 2/6 vs. 1/6; RETRIEVAL_INTEGRITY 0/3 vs. 0/3). Workers given no filter escalate rarely and largely accurately at this task scale; the folk premise that naive escalation means interrupt spam: which KG2's ratio-form gate encoded, was empirically false here, making that gate structurally unpassable against a zero-noise baseline (a gate-design finding the confirmatory study absorbs by using absolute caps). Sophistication actively hurt: this result sets the honest competitive floor for any successor architecture, and S2 is a mandatory head-to-head arm in the confirmatory study with a pre-committed honesty clause, if S2 dominates, the paper says so in the results, not in a footnote.

Self-injury. The architecture also harmed runs with nothing wrong in them: S5 clean-cell success was 3/12 against batch's 9/12, and seven of the nine clean failures were sentinel-induced, five escalation-cap grinds (the run's lifetime consumed adjudicating noise) and two replan-churn deaths in which result-carryover was empty because the false interrupt arrived before any worker had finished anything: the cost of replanning was repaid with nothing. The false-interrupt anatomy is uniform: all 34 orchestrator-level interrupts in the treatment arm had been judge-approved; 18

Table 1. Rival explanations for the missed detections, with adjudicated verdicts and discriminating evidence.

Rival story for the misses	Verdict	Discriminating evidence
Telemetry/normalization loss: evidence reached the wire and died before the matcher	**Refuted**	9/12 starved cells show *zero* raw post-injection surface observations; nothing existed to lose
Trajectory distortion: false interrupts replanned the surface out of the live plan	**Partial**	The strict signature appears in 0/12; but 7/12 were noise-consumed deaths, 1 was a replan-recompile coverage drop, and same-seed baselines re-visited the surface in 12/12
Semantic insufficiency: predicates cannot express the violation; re-observation returns ambiguous blobs	**Partial**	Predicate-too-weak in 2/12; the ambiguous bin is empty; all 8 content-shaped misses are deterministically decidable by fingerprint or field re-read, no LLM required
Base-agent/horizon failure: runs died before the surface was reachable	**Refuted**	Died-before-oracle: 0/17

were unattributable to any injection, 14 of those rode field-absence or schema-shape evidence on healthy traffic, and 3 fired on entirely clean runs. An uncalibrated monitor did not merely fail to help; it actively injured healthy runs: the literal form of the maxim that a sentinel producing bad tripwires is worse than no sentinel.

6 Failure Archaeology and the Adjudicated Diagnosis

Chain of death. Each of the 27 injected treatment cells was classified along the detection chain *compile* → *fire* → *escalate* → *judge* → *pause* → *attribute*. Of 18 misses: 1 had no covering tripwire compiled (L1); **12 were compiled-but-never-fired (L2); the dominant class**; 2 fired but were never escalated; 2 were escalated and ruled noise by the judge; 1 injection never fired. The architecture's failure was not blindness at compile time (coverage misses ≈ 0 once a heuristic artifact was corrected); it was silence at observation time.

Adversarial adjudication. Rather than self-grading the diagnosis, we sent it with the redesign specification to two frontier-model red teams for blind review, then adjudicated every trace-testable [FATAL]/[MAJOR] claim empirically with a deterministic replay battery over the banked corpus: pre-commitments (inherited thresholds, new gate rationales, the corroboration policy, the break-even model form) committed before any battery result was readable; total LLM cost of the battery: $0. The battery's foundation: all 27 injected worlds replay byte-identically from their configurations. Eighteen claims were dispositioned, roughly half holding and half refuted per rater: the empirical justification for *adjudicating* external review rather than deferring to it or dismissing it.

The diagnosis, in three findings. *(Findings, not laws: empirical regularities of this setting with antecedents in partial observability and monitorability theory.)*

(1) **Compilation is information-bounded.** A compiler cannot monitor a surface it was never told exists; mechanically supplying the environment surface lifted paper coverage from 67% to 89%. Probes inherit this bound through the compile dialect.
(2) **Detection is observation-bounded, and here, self-inflicted.** A correct, armed monitor detects nothing on a surface nobody observes post-violation. The original diagnosis blamed the world's geometry (single-visit surfaces); the same-seed trajectory analysis corrected this: baselines re-observed every starved surface (12/12). The sentinel's own noise economy

consumed the lifetime in which re-observation would have occurred. The architecture did not fail to solve the observation problem; it manufactured it.

(3) **Precision and recall couple through run lifetime.** Every false fire spends turns, attention, and lifetime that detection itself needed: one cell logged 172 post-injection fires on uninjected surfaces before dying at its escalation cap. One mechanism, four gate symptoms: noise starved recall (KG1), the credulous judge generated the noise (KG2), grinds and churn destroyed clean runs and waste (KG3, KG4's waste arm). The opening specimen (§1) is this coupling in a single cell.

The judge's post-mortem. The corpus settles what the judge tier contributed: across every escalation it ever saw, it approved 34/34, including all 18 false ones; the only two genuine surface signals it ever down-ruled, it ruled noise; and its highest-confidence intervention was approving, at 0.98, a hallucinated misreading of a file listing; the approval that destroyed sensor coverage twelve seconds before the injection in the opening specimen. The external attack "removing the judge loses its semantic fallback" was refuted in-corpus: there was no semantic fallback to lose. The judge supplied negative semantic value, and its retirement costs nothing the corpus can find, though we record the design-level residue that in worlds with weaker predicates the conclusion could differ.

Errata, kept visible. Two v1 diagnostic claims were corrected by the battery and remain in the record with their corrections: the "single-visit surface" starvation mechanism was withdrawn (the cell's passage was fetched only *post*-injection; the original narrative was inverted by its own trace), and "exactly one matcher defect" was corrected to a **dead-pattern class**, 84 armed URL patterns that could never match any world path under the live dialect, 61 host-qualified, 19 on covering tripwires, across 8 cells; an instrument-class defect repaired with a compile-time pattern-liveness regression.

The shadow replay (mechanism validation before construction). Deterministic probes were mechanically constructed for all nine pilot injections from each armed tripwire's target surface plus same-seed clean baselines, executed against re-instantiated mutated worlds, and the pre-committed corroboration policy was replayed offline over the pilot's actual 34-interrupt stream. Probe-primary corroboration blocked **18/18 false interrupts** while suppressing 15/16 true ones, and projected strict recall *rose* to 21/27, because probes recover what interrupts lose (RETRIEVAL_INTEGRITY 0/3 → 3/3 projected; per-category projection API 6/6, AUTH 6/6, RI 3/3, TC 4/6, SD 2/6). **Every number in this paragraph is a projected ceiling under stated assumptions (mechanically constructed probes the LLM compiler has not yet produced; fixed trajectories; the same nine leaked injections): an upper-bound feasibility test of the mechanism, not a prediction of system performance, and it is quoted nowhere in this paper as a result**. The replay also broke one v2 design clause as drafted (second-signal corroboration self-corroborates under noise storms; deleted) and measured the probe-overhead and perturbation constraints of §3.2. Six residual projected misses decompose into status-only probe bases (2), cadence starvation on short runs (3), and the never-fired injection (1): driving the event-gated cadence requirement.

> **[NEW · author review: exploratory failure analysis; concerns the Phase-1b (v2) verdict of §7, placed here as the v2 counterpart to §6's v1 archaeology].** The following is *exploratory and post-hoc*, drawn from the v3 cost-archaeology record; it localizes *where* the economic failure lives as input to v3 and does **not** re-open, soften, or relabel the OVERALL FAIL, which stands on 1bKG3.
>
> *(a) The KG3 overhead is monocausal: the per-plan compile.* Per-event trace cost-bucketing decomposes the 55.5% clean overhead (v2 clean median $0.3642 vs batch $0.2342, against the 12% cap) into a single net-new component: the once-per-run LLM compile-to-arm call (median $0.1376 on clean runs; the v2−batch clean delta is $0.130), present in 12/12 clean cells. The detection substrate: probes, arming, corroboration, the D29 event-gated cadence, the D30 arm-time sweep, carries no measured cost and is near-free **under the benchmark's free-re-observation assumption** (a deterministic mock world; see the §10 mock-floor

caveat, which travels with this claim). The cost is thus localized to the one-time compile-to-arm step, not to runtime monitoring.

(b) The KG4 waste-parity miss is marginal and is not monitoring overhead. v2's median wasted work was 7,008 tokens vs heartbeat S3's 6,404: a 1.09× ratio, just past parity, and is entirely discarded worker rework of the same *kind* S3 produces (monitoring contributes no measured tokens). It concentrates in 8 of 31 non-clean cells (waste 7,291.5 with a discarded worker vs 6,495 without) and is uncorrelated with detection latency, replans, or discard count (Pearson r = 0.007, −0.061, 0.045 respectively), so it is not attributable to the monitoring apparatus.

(c) The KG2 clean-success gap is task-difficulty noise, and the FIR-0 result is a strong positive. v2 clean success (8/12) trailed batch (9/12) by one cell; all four v2 clean shortfalls were output-validation failures on the strict report-assembly tasks at *zero* interrupts: task difficulty, not monitoring cost. The false-interrupt outcome is robust where it was built to be: clean-cell median FIR 0 and maximum 0 false interrupts, and the legitimate-write case that motivated v2's write-surface policy (the D31 settings-migration write) was in the clean set and ran at zero interrupts, while the naive judge baseline S2 false-fired on four clean cells and v2 stayed silent on all of them. The redesign cured v1's false-alarm pathology on exactly the case built to trip it, where a naive monitor still fires.

7 Confirmatory Study (Phase 1b)

The confirmatory study is the legitimate rerun path articulated *before* the pilot verdict existed: report v1 as it stands, diagnose from traces, change the design with cited evidence, and re-measure under fresh pre-registration. Its integrity rules are in ink:

Frozen gates, before any v2 data. All shared quantities inherit v1's pre-verdict thresholds verbatim (strict recall ≥60%; ≥50% in ≥4/5 categories; kill floor 40%; TTD ≥2×): no threshold derives from anything the diagnosis reported. New gates: detection (1bKG1) adds **probe validity as a hard gate** (a seeded 20% audit of probe-generated interrupts must show 100% targeted/fresh/non-perturbing/independent, with failures excluding that interrupt class from recall before the gate computes) and a **recovery-quality clause** (detect-only, detect-and-recover, and detect-and-justified-abort counted separately; at least half of strict detections must be the latter two, aborts cannot launder parity). Noise and self-harm (1bKG2) use **absolute caps**, the lesson of the unpassable ratio gate: clean-cell median FIR = 0, P95 ≤ 1, max ≤ 3 false interrupts per clean cell, zero escalation-cap grinds, pre-detection false-interrupt budget ≤ 2 on injected cells, and an absolute clean-success floor (≥60% and within 10 points of batch). Economics (1bKG3): clean overhead ≤12% with probe costs booked as waste, unamortized per-run as the primary view. Versus heartbeat (1bKG4): TTD ≥2× *and* wasted-work parity-or-better with probe costs included, with a sensitivity row censoring undetected cells at run end. An instrumentation-integrity replay (raw-trace-to-label, prototyped by the battery) runs on 100% of injected cells before any gate computes.

Anti-overfit structure. Injection fire counters and payload parameters are re-drawn from fresh seeds the designers never see; retreat-condition cells are labeled ex ante; and two **held-out categories** (§4): authored after v1, manipulation-qualified against the batch baseline, parameters escrowed with a non-implementer and verifiable against a public hash, sit inside the recall denominator, structurally unseen by the v2 design. A data embargo (no benchmark-world output observed before gate freeze) and a dev-run ledger (every execution involving any v2 component, logged) stand throughout. The compiler that produces v2's monitors is itself category-blind (§3.3): it receives no failure-category list and is grounded only by seen-category examples chosen under a rule frozen before tuning, so the held-out categories are unseen in the very vocabulary the compiler reasons with, not only in their escrowed parameters. A custody rule keeps the recall denominator honest: the held-out load-bearing-assumption count is computed escrow-side and never fed back into compiler iteration. One pre-registration obligation remains owed before any Phase-1c data

exists: the probe-failure policy (retry budget, persistence threshold, transport-versus-world fault classification) is committed as a numbered deviation before that data is seen.

> **[NEW · author review: held-out custody note]** The held-out injection instances were drawn by an automated procedure using a cryptographic random source with no seed and a single-draw guard; no author wrote them. The held-out blinding surface is a single sealed file whose SHA-256 is committed in the repository and verified by the loader at launch. During custody transfer the lead author incidentally viewed the opening few lines of this sealed file for a few seconds and did not read its contents. The compile is category-blind by construction, with no per-category detection path, which is verifiable by inspection of the frozen prompt, and it was created and frozen after the transfer; its general extraction repertoire is independently motivated by the seen categories. The integrity of the held-out generalization test therefore rests on the inspectable category-blindness of the frozen compile together with the unseeded cryptographic draw, and does not depend on author non-exposure alone.

Arms and reporting rules. Two-tier v2 is the designated primary arm, committed before the diagnosis battery's results were readable: there is no post-selection between it and the rebuilt-judge arm, which runs as exploratory only. Baselines: S1 batch, S3 cost-matched heartbeat, and S2 mandatory head-to-head under the honesty clause (§5), if the naive policy dominates v2 on recall at comparable noise, that is a results-section sentence, not a footnote. Category-level clauses are reported with Wilson lower bounds wherever n is small, and any category with $n < 3$ is descriptive only. Phase 1 posteriors enter v2 solely as weakly-informative priors, labeled exploratory, with no gate consuming them: passive-paradigm outcomes cannot calibrate an active-paradigm system, but discarding all signal is worse; the waiver is written. The boundary between instrument and system is maintained: the dead-pattern matcher repair (§6) is instrument-class, fixed with regression evidence and deviation-logged; every other change is system-under-test, existing only in v2 and measured only by these gates.

Results (Phase 1b, deviation-finalized).

> **[NEW · author review]** The confirmatory matrix ran to completion: **172/172 cells banked**, 43 cells/arm × 4 arms (S1 batch, S2 naive, S3 cost-matched heartbeat, V2 two-tier sentinel), over 21 original-injected + 10 held-out + 12 clean per arm, and the gates computed once on the complete matrix. The deviation-finalized verdict (provenance: sealed `gate_report.json` → audit-folded `gate_report_final.json` → the §10 instrumentation-replay correction logged as deviation D34, plus the pre-registered TTD run-end censoring rule of decision_memo_phase1.md §4):

```
1bKG1 detection (four pre-registered sub-terms, prereg_1b.md §2): recall 10/15
    = 66.7% (>=60%, kill floor 40% clear); categorical 5/5 seen categories
    >=50% (API_SURFACE 6/6, SCHEMA_DRIFT 3/3, PERMISSION_AUTH 6/6,
    TOOL_CONTRACT 3/3, RETRIEVAL_INTEGRITY 3/3); probe-validity audit (seed
    1102) 10/10 targeted/fresh/non-perturbing/independent; recovery-quality
    17/24 = 70.8% (detect-and-recover 3 + justified-abort 14)            -> PASS
Standing precondition (SEPARATE admissibility check, prereg_1b.md §2 / memo §4;
    runs BEFORE the gates, NOT a 1bKG1 sub-term): instrumentation-integrity
    replay 123/124 byte-identical; the 1 non-identity is benign
    (a1-S3-quota_cliff-s7968: order-only quota_remaining swap, status-identical,
    non-detection field, non-detecting S3 arm), characterized + logged as D34
    (artifact instrumentation_replay_audit.json); all five V2 RB cells
    byte-identical, generalization unaffected                  -> precondition OK
  [NOTE: matrix_gates.py FOLDS this precondition into the 1bKG1 boolean, so the
   non-identity forces 1bKG1=FAIL in gate_report_final.json even though all four
   detection sub-terms PASS. This DIVERGES from the prereg, which lists the
   replay as a SEPARATE Standing precondition, not a 1bKG1 term. Reported here
   decomposed: detection sub-terms PASS; Standing precondition OK (1 benign
```

```
    non-identity). The OVERALL FAIL is unaffected -- it rests on 1bKG3.]
 1bKG2 noise/self-harm: clean median FIR 0, P95 0, max 0 false interrupts;
     0 escalation-cap grinds; pre-detection median FIR 0 (cap 2);
     clean success 8/12 = 66.7% (>=60% and within 10pp of batch 75%)      -> PASS
 1bKG3 economics: V2 clean median $0.3642 vs batch $0.2342;
     clean overhead 55.5% (cap <=12%)                                     -> FAIL
 1bKG4 vs heartbeat: FAILS on wasted-work parity -- V2 7008 vs S3 6404 tokens
     = 1.09x (machine-computed, not parity-or-better). The TTD arm resolves for
     V2 only via the PRE-REGISTERED run-end censoring of S3's undetected cells
     (decision_memo_phase1.md §4 sensitivity rule, NOT an instrument-class
     deviation; S3 detected 0/31); the conjunction is dead on waste-parity
     regardless of the TTD arm                                            -> FAIL
 OVERALL                                                                  -> FAIL
```

The verdict is **FAIL, driven by 1bKG3 economics**: strong detection that fails the efficiency bar.

Detection inventory (ledger). V2 detected **24/31** injected cells; the passive baselines were inert (**batch S1 0/31, heartbeat S3 0/31**) and naive **S2 12/31**. On the five *seen* categories V2 was perfect: API_SURFACE 6/6, PERMISSION_AUTH 6/6, SCHEMA_DRIFT 3/3, TOOL_CONTRACT 3/3, RETRIEVAL_INTEGRITY 3/3, recovering exactly the content-shaped, single-visit categories (RETRIEVAL_INTEGRITY, TOOL_CONTRACT) that were invisible to v1's passive predicates (§5). **Generalization to held-out:** V2 detected the held-out **RESOURCE_BUDGET 3/5** where every baseline scored 0/5, genuine transfer of the category-blind change-shape compiler to a structurally-unseen category, while the second held-out category, **DEPENDENCY_VERSION (silent minor-version pagination truncation), was missed by every arm including V2 (0/5)**. The trace mechanism (determined read-only over the DV cells) is **observational, not a compiler-coverage gap**, and is shared by every arm: reported as a detection/benchmark limit, not laundered. V2 *did* arm the load-bearing listing surface (`/repo/files`) and capture its arm-time clean baseline in all 5 cells, so the miss is **not** information-bounded; the injection is instead observationally weak across the harness: the version label surfaces in no observed payload (no surface carries a version field), and its only proxy, a default-page-size truncation, is overridden whenever the plan passes explicit pagination parameters (the §4 *plan-time pre-armoring defeats default-drift* texture) and is a no-op when the collection already fits the new page size, so on most seeds the armed surface reads byte-identical pre- and post-injection. On the lone seed where the truncation did surface on the worker channel (a 5→3 listing), V2's event-gated pre-completion sweep re-read only `/repo/gate_status` and never re-diffed `/repo/files` against its baseline: the **observation-bounded** mechanism of §6 (Finding 2) recurring on the probe channel (the §10 projection-optimism residual: the live sweep can fail to re-cover an armed surface). That every arm missed 0/5, including the anomaly-escalating naive baseline S2: confirms a silent, well-formed shorter listing carries no error or status signal for any monitor to catch: a detection/benchmark limit rather than a v2-specific generalization failure.

[NEW · author review: denominator reconciliation] These figures index different denominators and reconcile as a single chain, stated here so a reader need not assemble them across the section. Of the **31 injected cells**, v2 detected **24 → 77%**. The pre-registered recall *gate* (1bKG1), however, is computed only over the **recoverable class**: the injected cells that admit an oracle recovery path under §4's labeling (the complement, retreat-condition cells, are scored for abort quality, not recovery), which numbers **15 cells**, of which v2 detected **10 → 66.7% gate recall**. The 77%-detected-versus-67%-recall difference is therefore *definitional*, not a discrepancy: detection counts every injected cell, while the gate denominator is the recoverable subset. Those 24 detections in turn split by recovery quality into **detect-and-recover 3 + detect-and-justified-abort 14 + detect-only 7 = 24**, with **17/24 = 70.8%** in the two passing buckets (aborts counted separately, so they cannot launder parity).

Per-category recall with Wilson 95% lower bounds (seen categorical clause, small n): API_SURFACE 6/6 (W.610), PERMISSION_AUTH 6/6 (.610), SCHEMA_DRIFT 3/3 (.439), TOOL_CONTRACT 3/3 (.439), RETRIEVAL_INTEGRITY 3/3 (.439); three categories sit at n=3, where a single cell swings 33 points (the G13 caveat, reported with the bound).

> **The v1 failure, reversed (1bKG2).** v1's signature pathology: a judge-rubber-stamped false-interrupt economy that starved recall and injured clean runs (median FIR 1.0; §5), is **eliminated**: clean-cell median FIR 0, P95 0, maximum 0 false interrupts, zero escalation-cap grinds, and a pre-detection false-interrupt median of 0 on injected cells. v2 buys back recall *and* drops noise to the naive baseline's zero floor.
>
> **Recovery quality.** Of 24 strict detections: 3 detect-and-recover, 14 detect-and-justified-abort (17/24 = 70.8% in the two passing buckets), 7 detect-only; aborts are counted separately and cannot launder parity.
>
> **S2 head-to-head (honesty clause).** Naive S2 detected 12/31 at clean median FIR 0; v2 detected 24/31 at the same zero-noise floor and dominated S2 on recall, uniquely detecting RETRIEVAL_INTEGRITY 3/3, TOOL_CONTRACT 3/3, and RESOURCE_BUDGET 3/5 (all S2 0). Unlike the §5 pilot, sophistication helped here, but the help did not survive the economics gate.

Holdout qualification (already complete, reportable now). Both held-out categories passed manipulation qualification against the batch baseline: RESOURCE_BUDGET on its primary host at clean 3/3 / injected 3/3-fail on first attempt; DEPENDENCY_VERSION qualified at its third specification revision (clean 3/3; injected 3/3 failing by the intended truncation wound, each failure carrying the wound's signature in ground-truth world state), with the two failed revisions and their defeat mechanisms reported as benchmark findings (§4) rather than discarded.

8 Real-Suite Study

[PENDING: governed by schedule gate 3: a frozen minimum of 50 validated onboarded tasks from established suites (GAIA [31], tau-bench [32], SWE-bench Verified subsets) by 2026-08-31, with pre-committed descope branches: 30–49 validated tasks narrows to a single-suite study; fewer than 30 retargets the publication plan entirely. This section will carry: onboarding and validation protocol (manipulation qualification at suite scale), the held-out-ontology arm (compilation under a disjoint failure taxonomy; the circularity answer), the natural-failure arm (no injection; naturally occurring invalidations annotated post-hoc; the external-validity answer), the fan-out arm (the only legitimate vehicle for any v2 efficiency claim, §9), and 5-seed results with paired bootstrap CIs. Nothing in it exists yet; no number from the synthetic pilot generalizes to it except through the break-even model's fitted form.]

[NEW · author review] Phase 1b is now resolved (§7); §8 remains the future real-suite study (schedule gate 3, a verdict-ready corpus by 2026-08-31) and is the **only** legitimate vehicle for any positive v2 efficiency claim: the fan-out arm where the break-even crossover variable actually varies (§9). The Phase-1b economic failure does not retarget §8; it sets §8's central question.

9 Economics

The architecture is cost-positive when `C + J + p\{}textperiodcentered{}R < p\{}textperiodcentered{} - W_sent)`: compile cost, judge cost, and expected replan overhead against the expected waste differential, scaling with fan-out and horizon. The model predicted payoff in high-fan-out, long-horizon, volatile regimes and failure for short tasks in stable environments; the pilot landed in the predicted below-crossover regime, and KG3 failed accordingly.

The headline cost numbers frame the gate failure: on injected runs the full architecture's median total cost was $1.179 against batch's $0.341, while *succeeding less often* (4% vs. 22%): paying 3.5× for worse outcomes, and clean-run overhead exceeded its pre-registered 12% cap. KG3's pre-committed branch demanded the break-even fit rather than a narrative, and the fit returned the harshest verdict the model can express. With the model form committed before fitting, parameters estimated from the 195-cell corpus (C = $0.322 median compile; J = $0.048 median judge; R = $0.257 median per-replan) yield a *negative* waste differential: W_batch = $0.145 vs. W_sent = $0.216: **v1's interrupt machinery wasted $0.072 more per injected run than the batch baseline it existed to save, leaving nothing for overhead to amortize against; there is no crossover at any fan-out,**

with probability 1.00 across 1,000 bootstrap resamples. The v1 efficiency claim is killed in ink, and the failure is reported as the central economic result, not as a caveat. The deeper reading: detection speed (KG4's surviving 3×) did not convert into savings because the interrupt machinery's noise costs, grinds, churn, replans into empty worlds, exceeded the waste it recovered. Speed is necessary for the economics; it is nowhere near sufficient.

What an honest v2 economics claim must now survive: probe overhead is real and measured (the naive cadence costs 16–22× the run's own tool traffic, 2,322 probe calls against a 107-call run at the extreme), so v2 books probe cost in the waste column under event-gated cadence, carries a ≤12% clean-overhead gate, reports unamortized per-run cost as primary (an amortized view over a pre-specified repeated-plan workload is reported descriptively only), and defers any positive efficiency claim to the full study's measured fan-out arm; the only place the model's crossover variable actually varies.

> **[NEW · author review]** The confirmatory economics fail exactly where the model predicted and exactly where v1 failed. v2's clean-run median cost was **$0.364** against batch's **$0.234**: a **55.5% clean overhead**, 4.6× the pre-registered 12% cap, so 1bKG3 fails and drives the OVERALL FAIL. Against the cost-matched heartbeat the §5 lesson repeated: detection speed did not convert to savings. v2's median wasted work was **7,008 tokens vs S3's 6,404: a 1.09× waste ratio, the wrong side of parity**, even though S3 detected nothing (0/31) and v2 detected the injected invalidation in a median of 9.5 tool calls; the TTD arm resolves trivially for v2 under the prereg's run-end censoring of S3's undetected cells, but 1bKG4 is a conjunction and fails on waste-parity. The redesign restored detection and eliminated false interrupts, yet its per-plan compile-to-arm step still spends more than the batch baseline it exists to save (the runtime probes being near-free only in this mock world; §10). **Runtime economics are the v2 failure and the v3 problem.**

> **[NEW · author review: correction applied].** The §9 economics sentence above has been re-attributed per the §6 cost autopsy: the overspend is the **per-plan LLM compile-to-arm step**, not the runtime probe-and-sweep machinery, which carries no measured cost **under the benchmark's free-re-observation assumption** (§10 mock-floor; the probes being near-free is a benchmark-floor artifact, not a deployment claim). The correction flag is resolved.
>
> **[NEW · author review: v3 problem, calibrated].** The §6 autopsy makes the v3 economic target concrete and bounds its difficulty. The dominant, fixable lever is the per-plan compile: a compile under ≈$0.028 (12% of batch's $0.234 clean median) would clear the benchmark's overhead cap. This is held against the central tradeoff (§10): the compile is the capability-critical step, so cheapening, caching, or conditioning it risks degrading detection and generalization; the economic fix is not free of detection risk. Two further constraints are named, not solved: v3 must design for the **real probe cost the mock hides** (§10), since probe volume and sweep frequency that read as free here are live overhead, and sweep-frequency tuning is in any case the weakest cost lever, because the waste failure is uncorrelated with sweep cadence (§6 cost autopsy). Finally the **write-surface information-theoretic residual** is named rather than fixed: a legitimate write and an injected permanent change, each leaving no independent expected state, are indistinguishable to a read-side detector (deviation D23; §10). v3 is therefore a fresh confirmatory cycle: new pre-registration, new held-out draw, not a patch.

9.1 Phase 1c: the fan-out arm (pre-registered; results forthcoming)

> **[NEW · author review: design and rationale only; the fan-out arm has not been run and no result of it is reported here].**
>
> *(a) Why 1b's economics failed in the regime the model predicted.* The cost autopsy localizes the 55.5% clean overhead to a single fixed charge: the once-per-run, per-plan LLM compile-to-arm step (median **$0.1376** clean), which accounts for essentially all of the V2−S1 cost delta and is incurred whole regardless of how much parallel work the plan spawns; the per-call census finds exactly one compile, zero recompiles, and $0 on the probe/sweep/corroboration substrate, so the autopsy's bottom line is a **design cost of the capability-critical compile, not an implementation defect**. At the pilot's emergent fan-out (~3

workers) that fixed compile amortizes over almost no parallel work, which is why the overhead is structural rather than a bug to be patched.

(b) The regime reframe. Because a fixed per-plan cost amortizes over fan-out, the economics question is really a **fan-out-regime question**. Re-fitting the committed break-even model to the 1b v2 estimates gives a *positive* waste differential (ΔW at fan-out 3 = **+$0.068**, unlike v1's −$0.072), which yields an **extrapolated** break-even fan-out of **N ≈ 86, 40, 25** at fault rates p = 0.10, 0.25, 0.50. That band sits **above the supervisor regime (~3–10 workers) and inside the swarm regime (~20–100+)**. These are model extrapolations from 1b data: *a prediction to be tested, never a measured Phase-1c result.*

(c) The pre-registered arm. Phase 1c is pre-registered (un-pinned draft) to test that prediction by forcing the worker count **N** as a controlled variable. Its estimand is **width-scaled**: fixed work per worker, total work growing with N, over N ∈ {1, 3, 8, 16, 32, 64}, set by the harness, not LLM-chosen. The primary quantity is a paired, quality-qualified, **persistent** net-cost crossover: the smallest grid fan-out at which the sentinel *would be* cheaper in expectation **and remain so at every larger tested N**, admitted only behind a quality floor (clean-success non-inferiority, recovery-quality, detection non-inferiority), and reported per fault rate p alongside the break-even fault probability $p^*(N)$ at every tested N. Integrity is gated **separately** from results: a manipulation-validity gate (1cKG0, correct width, fault-before-read, the aggregate actually poisoned, batch wounded at every N) is computed first as the only invalidator, while detection-and-recovery **scaling**, recall(N), coverage, FIR, is treated as a *reported outcome that feeds the economics, not a pass/fail validity check*, under four pre-committed, uncertainty-gated verdict branches over where (if anywhere) a crossover would land. We will measure these; we report none of them here.

(d) The purpose-built benchmark, qualified. Because the pilot's task could not supply a clean width-scaling, a dedicated benchmark was designed and qualified. N workers each read one immutable evidence shard and emit a demand-normalized allocation share against a **shared denominator** T = Σ d_i; a single silently-corrupted shard changes T, so **all N outputs become mathematically invalid at once: "all N wasted" is an arithmetic fact, not a harness policy**. By construction the only sound detection path is a **per-surface baseline-diff**: the mutation is plausible, internally consistent, and exposes no aggregate or cross-surface invariant, so detection means *finding the one mutated shard among N* (the single-surface detection-locus invariant). The benchmark is **qualified**: an arithmetic window sweep confirms that a single-shard mutation which is simultaneously a hard arithmetic wound **and** statistically non-anomalous exists with wide margin at every N ∈ {8, 16, 32}, and a plain-batch (S1) qualification confirms the task is constructible and wounds correctly, clean **9/9** exact-pass and injected **9/9** wounded by a global-total mismatch with all N outputs invalidated. **These are world-and-batch-baseline qualification facts; no sentinel (V2) has been run on the task, and no fan-out, recall(N), crossover, or economics result exists.**

[NEW · author review: Phase-1c scope and limitation]. Phase 1c is a **controlled synthetic scaling test of the monitoring mechanism on a constructed supervisor substrate**, not a swarm deployment and not the real-suite study, and **it has not been run**. No efficiency claim is made or implied by its design; a positive future result on this surface would be *necessary-not-sufficient*, with the deployment-efficiency claim remaining deferred to the real-suite fan-out arm (§8). The **mock-floor** still binds: probes are free in the mock world, so any net-cost crossover the arm might later **measure** would be an *optimistic lower bound* on cost-effectiveness, with live probe-volume(N) the deployment-cost canary (§10).

10 Threats to Validity

Construct. Injected invalidations approximate but do not sample the natural distribution; manipulation qualification guarantees each injection wounds the baseline, not that it is representative. The full study's natural-failure arm is the designed mitigation. The benchmark's content payloads are authored to flip cleanly under deterministic fingerprints; real-corpus contradiction detection may not reduce to an anchored hash.

[NEW · author review: lens-extraction bound] Content-contradiction detection additionally requires the compile to assign a content lens to the affected surface. This is reliable when the orchestrator's plan names and fetches the passage concretely, and less reliable under plans that name retrieval only generically,

where the compile may assign a structural lens and miss a content-only swap. This bounds the compile's extraction under generic plans rather than the detection architecture itself, and it sits alongside the clean-fingerprint content payloads noted above.

Internal. LLM stochasticity at every tier; the pilot ran n=3 seeds by design and wrote its gates on medians; plan-shape variance (orchestrator consolidation, §4) is real and uncontrolled by construction. Attribution is rule-based with a mandated audited sample (13/13 agreement).

External. One harness, one model family, one mocked world at pilot scale; 8–26-call tasks sit below the break-even crossover by the model's own arithmetic, so pilot economics generalize only through the fitted model, never directly.

Benchmark information leak into the redesign. v2 was designed from the pilot's traces on these nine injections. Mitigations: fresh parameter draws the designers never see, the escrowed held-out categories inside the confirmatory recall denominator, and the full study's held-out-ontology and natural-failure arms; the leak is acknowledged here rather than laundered. **[NEW · author review: pointer]** The held-out custody and blinding statement (§7) details the unseeded cryptographic draw, the single committed-hash sealed-file surface, the incidental brief view during transfer, and why integrity rests on the inspectable category-blindness of the post-transfer frozen compile rather than on author non-exposure.

Projection optimism. The shadow replay's 21/27 is an upper bound computed under stated assumptions, with mechanically constructed probes the compiler has not yet been asked to produce, on the same nine leaked injections, with no trajectory feedback and no probe-side false-interrupt measurement. It validates a mechanism's feasibility, not a system's performance, and appears nowhere in this paper as a prediction.

[NEW · author review: v0.4 correction; supersedes the v0.3 wording preserved below]. The Phase-1b instrumentation-integrity replay (Task-A byte-identity, 100% of injected cells) ran on **124 injected-class cells: 123 PASS / 1 FAIL**. The single non-identical cell is **a1-S3-quota_cliff seed s7968** (RESOURCE_BUDGET, S3 arm); the other four a1-S3-quota_cliff seeds (s2594, s5682, s6570, s6820) are byte-identical. The diff is **not** a "+1 in quota_remaining": it is a **net-neutral ORDER SWAP** of the in-body `quota_remaining` value across two adjacent metered calls, status-identical (200→200), body-only: counter 12 `GET /pricing/quote/WID-001` recorded 11 → replayed 12, and counter 13 `GET /shipping/rates/THM-002?dest=us-east` recorded 12 → replayed 11. **`detection_relevant=False`** for both: the matcher consumes status + body predicates, `quota_remaining` is the in-body mirror of the `X-Quota-Remaining` header (`world/server.py:576`), is not a tripwire target, and headers are not compared, so no matcher verdict, metric, or gate quantity depends on it. The replay is **deterministic** (3/3 identical passes), so it does not introduce the non-identity. The recorded pair is non-monotonic: counter 12 reads 11 while the later counter 13 reads 12, impossible under the in-counter-order decrement the replay performs, which points to a live-capture origin: given the **unlocked** decrement (`world/server.py:571-576`, no lock), the consistent explanation is two concurrent metered requests resolving out of counter order in the live run, which the in-order replay **exposes** rather than causes. Reach check: all **five** V2 a1/quota_cliff (RESOURCE_BUDGET) cells byte-match (mismatch=0); the RB 3/5 generalization result sits on **no** replay-failing cell. Logged as deviation **D34**; forward fix = **serialize the live world's quota decrement** (`world/server.py`), not "under the replay harness."

[NEW · author review: projection optimism, now operationalized]. v2's content/value probes need a clean pre-violation baseline; in the shadow replay (§6) that baseline was the same-seed clean world the offline harness already held; an affordance the live system does not get for free. v2 generates it with the arm-time side-channel sweep (§3.2); the residual risk is that the live sweep's earliest-clean capture can lag a surface first touched only after injection, which the projection's same-seed shortcut hid. The 21/27 shadow projection stays an upper bound and is quoted nowhere as a result.

[NEW · author review: relation-detection residual]. Content-contradiction detection requires both that the affected surface be **armed** and that the compile assign a **content/value lens** to it (the

D33 lens-extraction bound, above). A further gap: relation-breaking changes; a cross-surface invariant violated without any single surface changing shape, are not yet expressed by the six change-shapes, so relation-class faults can be armed-but-unlensed and missed. The §7 DEPENDENCY_VERSION miss is a *distinct* shape and is cross-referenced here for contrast: there the surface *was* armed but the change was observationally null on most seeds and, where it surfaced, the probe sweep did not re-cover the armed surface; an **observation-bounded** gap (§6 Finding 2), not a relation-shape or content-lens gap.

[NEW · author review: write-surface information-theoretic residual]. Detection is information-bounded on surfaces the agent never re-reads; the dual holds on the write side; an agent's own write can overwrite a world change it never observed (the WRITE-SIDE SINGLE-VISIT finding, deviation D23: a pre-drift read, the injection, then a migration write that clobbered the drift sight-unseen). v2's read-side arm-time sweep does not cover write-only surfaces; closing this is a v3 instrumentation question.

[NEW · author review: mock-floor / external validity]. v2's near-free detection substrate (§6 cost autopsy) is an artifact of the benchmark world: it is a deterministic mock in which re-observation is free, so probes and sweeps incur no measured cost. In a live deployment, probes re-fetch real endpoints and re-read real files at real monetary cost and latency, so the measured 55.5% clean overhead is a **lower bound**, and probe volume and sweep frequency: cost-free in the mock, would re-emerge as real overhead. The compiler's family-arming over-provisioning is the canary: a median run arms many more probes than it ever exercises (e.g. API_SURFACE armed 35 vs 19 exercised), $0 here but a live cost in deployment. Economic viability in deployment is therefore an open question this benchmark cannot settle.

[NEW · author review: cost/capability tradeoff]. The one cost lever the autopsy localizes (the per-plan compile, §6) is also the capability-critical step: it is the LLM reasoning that produces detection coverage and the category-blind generalization (§7). The obvious economic fix: a cheaper or shrunken compile, therefore risks degrading detection and generalization; the economic fix is not free of detection risk.

[NEW · author review: KG4 marginality]. 1bKG4's waste-parity failure (1.09×) is marginal and, per the §6 cost autopsy, is discarded worker rework uncorrelated with the monitoring apparatus rather than monitoring overhead; it does not alter the OVERALL FAIL, which rests on 1bKG3 economics.

Gate-form pathology. Ratio-form gates against an unexpectedly quiet baseline are structurally unpassable (KG2 vs. S2's zero noise); the confirmatory gates use absolute caps wherever the comparison quantity is itself under test.

Conclusion validity / builder bias. The benchmark, treatment, and gates share authorship. The counterweights are the apparatus itself: frozen pre-registration, pre-committed branches, custody pins, binding external red-team adjudication with trace-level dispositions, a public escrow hash, and negative results reported at full prominence, including a naive baseline beating the proposed architecture (§5). The strongest evidence the apparatus binds is behavioral: it returned a kill verdict on its authors' own architecture and the branches executed as written.

Holdout-design limits. The held-out categories are held out in *parameters and presence*, not in kind: they were authored by the same hands under the same ontology, and the batch baseline defeated two of three designs before one qualified, evidence that authoring genuinely wounding faults is itself hard, and that surviving designs are selected for wounding *this* harness's planning behavior. The full study's held-out-*ontology* arm (compilation under a disjoint taxonomy) and natural-failure arm remain the principal generalization tests.

11 Conclusion

We pre-registered an architecture we believed in, and the gates we froze killed most of it: detection recall in the kill band, a judge that filtered nothing, economics inverted against the baseline. The apparatus held: branches executed as written, negative results reported at the same prominence as the survivor, and because every run was fully traced and byte-replayable, the failure yielded a mechanism rather than a shrug: in budget-bounded execution, monitoring precision and recall share

the run's finite lifetime, and an uncalibrated monitor manufactures the very observation starvation it cannot survive. The surviving 3× detection-latency result, the naive baseline's competitiveness, and the boundedness findings define the design space for any plan-compiled monitor. Whether the replay-validated redesign clears its frozen gates is an empirical question with a dated answer; this paper reports the verdict either way. **[NEW · author review]** The redesign restored detection and eliminated false interrupts, and missed its economics bar: under fresh pre-registration v2 detected 24/31 injected faults (recall 66.7% on the recoverable class), partially generalized to the held-out categories, clearing RESOURCE_BUDGET (3/5) where every passive baseline scored 0/5 but, like every arm, missing DEPENDENCY_VERSION (0/5), and eliminated v1's false-interrupt self-harm to a clean-run noise floor of zero, yet its 55.5% clean overhead breached the 12% cap, so the pre-committed verdict is **FAIL on 1bKG3 economics**: strong, generalizing detection at zero false alarms that does not yet pay for itself, which is exactly the problem v3 must solve.

Data Availability

A replication package: the benchmark world and harness, the frozen pre-registration with custody pins, all 195 pilot traces plus the deterministic replay battery, the qualification and adjudication records, the deviation log, the dev-run ledger, and the held-out-category escrow hash, is available at [ANONYMIZED-REPLICATION-LINK]. The held-out parameter values themselves remain escrowed with a non-implementer until the confirmatory study's gates compute; their SHA-256 is committed publicly so the file cannot be swapped.

References

[1] Execution monitoring survey (Dunlap et al.).
[2] Davis-Mendelow et al., Assumption-based planning, AAAI 2013.
[3] Bercher et al., Plan repair, ICAPS 2014.
[4] Bauer, Leucker, Schallhart. Runtime verification for LTL and TLTL. ACM TOSEM.
[5] Havelund, Rosu. Synthesizing monitors for safety properties. TACAS 2002.
[6] Ferrando, Cardoso. RVPLAN: Runtime Verification of Assumptions in Automated Planning. ICAART 2022, Vol. 2, pp. 67–77. DOI 10.5220/0010776500003116.
[7] Parallelized planning-acting. arXiv:2503.03505.
[8] Weak-to-strong monitoring. arXiv:2508.19461.
[9] AgentSpec. ICSE 2026. arXiv:2503.18666.
[10] Agent-C. arXiv:2512.23738.
[11] Pro2Guard. arXiv:2508.00500.
[12] ProbGuard. arXiv:2602.19844.
[13] LlamaFirewall. arXiv:2505.03574.
[14] MAST: multi-agent system failure taxonomy. arXiv:2503.13657.
[15] AgentOps. arXiv:2411.05285.
[16] Wasted-computation diagnosis in multi-agent systems.
[17] LumiMAS. AAMAS 2026. arXiv:2508.12412.
[18] DiLLS. CHI 2026. arXiv:2602.05446.
[19] Graph Harness. arXiv:2604.11378.
[20] SHIELDA. arXiv:2508.07935.
[21] Learning to Interrupt. arXiv:2604.06452.
[22] LangGraph interrupt documentation.
[23] Kephart, Chess. The vision of autonomic computing. IEEE Computer, 2003.
[24] Weyns. Introduction to self-adaptive systems. 2020.
[25] Nascimento et al. arXiv:2307.06187.
[26] ACM TAAS roadmap. DOI 10.1145/3686803.
[27] Anthropic Engineering. How we built our multi-agent research system.
[28] [ANONYMIZED: unpublished prior work by the authors; cite via anonymized supplement or omit at submission]
[29] [ANONYMIZED: unpublished prior work by the authors; cite via anonymized supplement or omit at submission]
[30] Zep temporal knowledge graphs. arXiv:2501.13956.

[31] GAIA. arXiv:2311.12983.
[32] tau-bench. arXiv:2406.12045.
[33] Karnofsky. Carnegie Endowment.
[34] Ferrando, Cardoso. RVPLAN: a general purpose framework for replanning using runtime verification. VORTEX@ISSTA 2021, pp. 22–25. DOI 10.1145/3464974.3468447.
[35] Bozzano, Cimatti, Roveri, Tchaltsev. A comprehensive approach to on-board autonomy verification and validation. IJCAI 2011, pp. 2398–2403.
[36] Bensalem, Havelund, Orlandini. Verification and validation meet planning and scheduling. STTT 16(1):1–12, 2014.
[37] Ferrando, Cardoso. Runtime monitoring of action specifications for replanning in classical planning. VORTEX 2025.